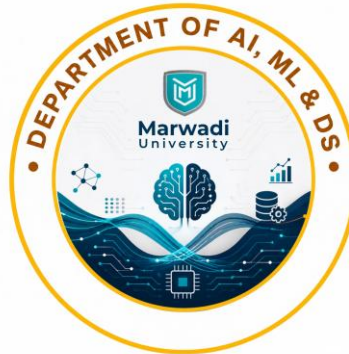




Marwadi
University
Marwadi Chandarana Group



LABORATORY MANUAL

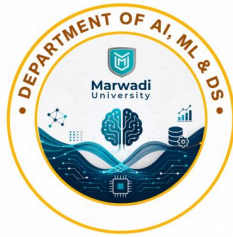
PROGRAMMING WITH PYTHON

Course Code: 01AP0303 | Credits: 4 | Practical: 2 Hours/Week

Student details	To be completed
Name	
Enrollment No.	
Semester / Class	
Laboratory Batch	
Academic Year	
Faculty	

Department of B.Tech CSE-AI/ML/DS-PBL
Faculty of Engineering and Technology
Marwadi University, Rajkot

Laboratory Record Certificate



Marwadi
University
Marwadi Chandarana Group



This is to certify that _____, Enrollment No. _____, has satisfactorily completed the prescribed laboratory work for Programming with Python (01AP0303) during Academic Year 2026-27.

Role	Name / Signature	Date
Course Faculty		
Laboratory Faculty		
Head of Department		

Student Declaration

The work recorded in this manual has been performed and understood by the student. External code, data, APIs, libraries and AI-assisted contributions, if any, must be acknowledged. Credentials, personal data and unauthorized content must not be included in submitted files.

General Laboratory Instructions

- Attend the laboratory punctually and complete each experiment during the scheduled session.
- Create a separate folder for each experiment and use meaningful filenames.
- Write and test the program independently; do not submit code without understanding it.
- Use comments, meaningful identifiers, modular functions and appropriate exception handling.
- Record the algorithm, source code, test cases, output evidence and conclusion.
- Never store passwords, API keys or private personal data directly in source code.
- Maintain academic integrity and acknowledge external or AI-assisted material.
- Take a backup of code and data after every laboratory session.

1. Institutional and Programme Framework

1.1 Institute Vision

To foster an environment that empowers people, organisations, and societies through education, ideas, research, and training.

1.2 Institute Mission

- To provide quality education and thereby bring social transformation.
- To create leaders through innovation and entrepreneurship.
- To cultivate the culture of research advancements.
- To imbibe universal consciousness.
- To stimulate growth through industrial and international partnerships.

1.3 Department Vision

To empower innovators in Computer Science & Engineering through education, research, and ethical practices to create sustainable solutions with global impact.

1.4 Department Mission

- Deliver quality education in Computer Science & Engineering through modern, technology-enabled pedagogies and strong industry collaboration.
- Foster innovation, entrepreneurship, and interdisciplinary research to drive creativity and problem-solving.
- Promote ethical computing, sustainability, and professional responsibility to nurture socially conscious graduates.
- Prepare students for lifelong learning, global employability, and leadership in emerging domains such as AI, Data Science, Cloud Computing, Cybersecurity, and beyond.

1.5 Programme Educational Objectives

- **PEO1 Technical Excellence:** Graduates will design and develop secure, scalable, and efficient computing systems using strong foundations in Computer Science & Engineering.
- **PEO2 Career Readiness and Leadership:** Graduates will excel in global careers, research, or entrepreneurship while demonstrating leadership, teamwork, communication, and ethical responsibility.
- **PEO3 Innovation and Lifelong Learning:** Graduates will adapt to rapid technological changes, pursue continuous learning, and drive innovation through research and interdisciplinary collaboration.
- **PEO4 Societal Responsibility:** Graduates will apply computing knowledge responsibly to create sustainable, inclusive solutions that address societal and global challenges.

1.6 Programme Outcomes (GAPC V4.0)

PO	Title	Statement
PO1	Engineering Knowledge	Apply mathematics, natural science, computing, engineering fundamentals and specialization knowledge to solve complex engineering problems.
PO2	Problem Analysis	Identify, formulate, review literature and analyze complex engineering problems to reach substantiated conclusions with consideration for sustainable development.
PO3	Design/Development of Solutions	Design creative solutions and systems/components/processes that meet identified needs with due consideration for health, safety, whole-life cost, net-zero carbon, culture, society and environment.
PO4	Conduct Investigations	Conduct research-based investigations, including experimental design, modelling, analysis and interpretation of data, to provide valid conclusions.
PO5	Engineering Tool Usage	Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, while recognizing limitations.
PO6	The Engineer and the World	Analyze and evaluate societal and environmental aspects and sustainability impacts with reference to economy, health, safety, law, culture and environment.
PO7	Ethics	Apply ethical principles and commit to professional ethics, human values, diversity and inclusion, and applicable national and international laws.
PO8	Individual and Collaborative Team Work	Function effectively as an individual and as a member or leader in diverse and multidisciplinary teams.
PO9	Communication	Communicate effectively and inclusively through reports, design documentation and presentations while considering cultural, language and learning differences.
PO10	Project Management and Finance	Apply engineering management and economic decision-making to individual and team work and to multidisciplinary project management.
PO11	Life-Long Learning	Engage in independent lifelong learning, adapt to emerging technologies and apply critical thinking in the broad context of technological change.

1.7 Programme Specific Outcomes

PSO	Theme	Statement
PSO1	Software Systems	Design and develop secure, efficient and scalable software systems, intelligent applications and computing solutions using industry-standard tools and frameworks.
PSO2	Emerging Technologies	Apply Artificial Intelligence, Data Science, Cloud Computing, Cybersecurity and emerging technologies responsibly to create innovative and sustainable solutions for real-world challenges.

2. Course Framework

Item	Details
Course title	Programming with Python
Course code	01AP0303
Credits	4
Prerequisite	Programming Basics
Laboratory hours	2 hours/week; 24 hours total
Suggested practicals	12

2.1 Course Outcomes

CO	Course outcome statement	Bloom level
CO1	Apply Python programming fundamentals to solve computational problems effectively.	Apply
CO2	Develop modular and reusable Python programs using functions, files, and object-oriented concepts.	Apply
CO3	Build industry-oriented applications integrating databases, APIs, and automation tools.	Create
CO4	Analyze datasets and generate insights using Python data analytics libraries.	Analyze
CO5	Design and implement real-world software projects collaboratively using Python development tools.	Create

2.2 CO-PO-PSO Articulation Matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2	-	-	2	-	-	-	-	-	1	2	-
CO2	2	2	2	-	3	-	-	1	1	-	2	3	-
CO3	2	2	3	-	3	2	1	2	2	2	2	3	2
CO4	2	3	-	2	3	1	-	-	1	-	2	2	3
CO5	2	2	3	1	3	2	2	3	3	3	3	3	2

Correlation scale: 3 = Substantial (High), 2 = Moderate (Medium), 1 = Slight (Low), - = No direct correlation.

2.3 CO-PO-PSO Mapping Justification

CO	Primary PO linkage	PSO linkage	Rationale
CO1	PO1, PO2, PO5	PSO1	Applies programming and computing fundamentals to formulate and solve computational problems using Python tools.
CO2	PO3, PO5, PO11	PSO1	Develops modular and reusable programs through functions, files and object-oriented design.
CO3	PO3, PO5, PO8, PO10	PSO1, PSO2	Integrates databases, APIs and automation into industry-oriented applications and project workflows.
CO4	PO2, PO4, PO5	PSO1, PSO2	Uses analytics libraries to investigate datasets, interpret results and communicate evidence-based insights.
CO5	PO3, PO5, PO8, PO9, PO10, PO11	PSO1, PSO2	Designs, implements, documents and presents collaborative real-world software projects.

2.4 Experiment-CO Mapping

No.	Experiment	Mapped CO(s)
1	Smart Calculator	CO1, CO2
2	Employee Payroll Processing System	CO1, CO2
3	Student Grade Analyzer	CO1, CO2, CO4
4	Inventory Tracker	CO1, CO2
5	Log File Analyzer	CO2, CO4
6	Banking Account Management System	CO2, CO5
7	Customer Record Management App	CO3, CO5
8	Weather API Dashboard	CO3, CO4
9	Email Automation Tool	CO3, CO5
10	Sales Analytics Dashboard	CO4, CO5
11	Attendance Management System	CO3, CO5
12	Helpdesk Ticketing Tool (Capstone)	CO3, CO5

3. Assessment Rubric

Every experiment carries 10 marks using four components: on-time completion and submission (2), program execution and functional correctness (3), record/code quality and result documentation (2), and viva voce (3). The total obtained across 12 experiments is converted from 120 to 25 Term Work marks. Marks, assessment date and faculty signature must be recorded for every experiment.

Criterion	Maximum marks	Suggested scoring
On-time completion and submission	2	2 = on time; 1 = minor delay; 0 = late/not submitted
Program execution and functional correctness	3	3 = complete/correct; 2 = minor defect; 1 = partial; 0 = not working
Record, code quality and result documentation	2	2 = complete/clear; 1 = minor omissions; 0 = inadequate/absent
Viva voce	3	3 = clear mastery; 2 = basic understanding; 1 = limited; 0 = no meaningful response
Total	10	

3.1 Consolidated Evaluation Record

No.	Experiment	Time /2	Execution /3	Record /2	Viva /3	Total /10	Assessment date	Faculty signature
1	Smart Calculator							
2	Employee Payroll Processing System							
3	Student Grade Analyzer							
4	Inventory Tracker							
5	Log File Analyzer							
6	Banking Account Management System							
7	Customer Record Management App							
8	Weather API Dashboard							
9	Email Automation Tool							
10	Sales Analytics Dashboard							
11	Attendance Management System							
12	Helpdesk Ticketing Tool (Capstone)							
	Total obtained					/120		
	Converted Term Work marks					/25		

3.2 CSE Component - 20 Marks

The CSE component is divided into a Python/CSE MOOC course (10 marks) and development of a reusable Python library (10 marks). Each task must be assessed separately, with marks, date and faculty signature recorded below.

3.2.1 MOOC Course Assessment - 10 Marks

MOOC assessment task	Maximum marks	Marks awarded	Verification date	Faculty signature
Relevant Python/CSE MOOC selected and enrolled	1			
Progress/completion/certificate evidence submitted	3			
Learning summary and key concepts documented	2			
Application of MOOC learning demonstrated	2			
Source link/citation and viva explanation	2			
Total - MOOC course	10			

Required evidence: course title and link, enrollment/progress/completion proof, concise learning summary, application of learning, and individual explanation.

3.2.2 Reusable Python Library Development - 10 Marks

Develop a small reusable Python package for an approved purpose. Submit the source package, reusable functions or classes, documentation/docstrings, README, usage example or demonstration notebook, test cases and output evidence.

Python library development task	Maximum marks	Marks awarded	Review date	Faculty signature
Problem selection and library requirements	1			
Reusable functions/classes correctly implemented	3			
Package structure, modularity and code quality	2			
Documentation, docstrings and usage examples	2			
Testing, demonstration and viva explanation	2			
Total - Python library development	10			

3.2.3 Consolidated CSE Assessment Record

CSE task	Maximum marks	Marks awarded	Final verification date	Faculty signature
MOOC course	10			
Reusable Python library development	10			
Total CSE marks	20			

EXPERIMENT 01

Smart Calculator

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO1, CO2		

Aim

Develop a GUI-based calculator that performs arithmetic, percentage, and selected scientific operations with appropriate input validation.

Learning Objectives

- Apply variables, operators, functions and exception handling.
- Design an event-driven interface using Tkinter widgets.
- Validate user input and display meaningful error messages.

Software / Resources

Python 3.x, Tkinter, VS Code

Concept Brief

A graphical calculator links interface events to reusable functions. Tkinter provides Entry, Button and Label widgets, while the math module supplies scientific functions. Exception handling prevents invalid entries or division by zero from terminating the application.

Algorithm / Procedure

1. Create the main window, display field and button layout.
2. Append the selected digit/operator to the expression.
3. Evaluate arithmetic or percentage expressions; call scientific functions when selected.
4. Catch invalid expressions and division-by-zero errors.
5. Display the result and allow the user to clear the screen.

Reference Implementation

```
import tkinter as tk
from math import sqrt, sin, cos, radians

def press(value):
    display.insert(tk.END, value)

def calculate():
    try:
        expression = display.get().replace("%", "/100")
        result = eval(expression, {"__builtins__": {}}, {})
        display.delete(0, tk.END)
        display.insert(0, result)
    except (SyntaxError, ZeroDivisionError):
        display.delete(0, tk.END); display.insert(0, "Invalid input")

def scientific(operation):
    try:
        value = float(display.get())
        result = {"sqrt": sqrt(value), "sin": sin(radians(value)),
                  "cos": cos(radians(value))}[operation]
        display.delete(0, tk.END); display.insert(0, round(result, 6))
    except (ValueError, KeyError):
        display.delete(0, tk.END); display.insert(0, "Invalid input")

root = tk.Tk(); root.title("Smart Calculator")
display = tk.Entry(root, width=28, font=("Arial", 16), justify="right")
display.grid(row=0, column=0, columnspan=4, padx=8, pady=8)
keys = ["7", "8", "9", "/", "4", "5", "6", "*", "1", "2", "3", "-", "0", ".", "%", "+"]
for i, key in enumerate(keys):
    tk.Button(root, text=key, width=6, command=lambda k=key: press(k)).grid(row=i//4+1, column=i%4)
tk.Button(root, text="=", width=13, command=calculate).grid(row=5, column=0, columnspan=2)
tk.Button(root, text="C", width=13, command=lambda: display.delete(0, tk.END)).grid(row=5, column=2, columnspan=2)
for col, op in enumerate(("sqrt", "sin", "cos")):
    tk.Button(root, text=op, width=8, command=lambda x=op: scientific(x)).grid(row=6, column=col)
root.mainloop()
```

Student Tasks and Test Evidence

- Test addition, division, percentage, square root and trigonometric operations.
- Add one new scientific operation such as logarithm or power.
- Record one valid and one invalid-input output.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

Conclusion

Viva Questions

- What is event-driven programming?
- Why is lambda used in the button command?
- What is the purpose of try-except?
- Why should unrestricted eval() be avoided?
- How does grid() position widgets?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
GUI operation and functional correctness (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Algorithm, code readability and test evidence (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 02

Employee Payroll Processing System

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO1, CO2		

Aim

Create a modular payroll application to calculate gross salary, tax, overtime payment, deductions and net salary.

Learning Objectives

- Use functions and conditional statements for payroll rules.
- Validate numerical inputs.
- Generate a clear employee salary report.

Software / Resources

Python 3.x, Jupyter Notebook

Concept Brief

Payroll processing converts basic pay and allowances into gross salary, applies tax and other deductions, and calculates net salary. A modular design isolates each rule so that it can be tested and modified independently.

Algorithm / Procedure

1. Accept employee name, basic salary and overtime hours.
2. Calculate HRA, DA and overtime payment.
3. Calculate gross salary and determine income tax using slabs.
4. Subtract tax and provident-fund deduction.
5. Display an itemized payroll report.

Reference Implementation

```
def income_tax(gross):
    if gross <= 30000:
        return 0
    if gross <= 60000:
        return 0.10 * (gross - 30000)
    return 3000 + 0.20 * (gross - 60000)

def payroll(name, basic, overtime_hours):
    if basic < 0 or overtime_hours < 0:
        raise ValueError("Salary and hours cannot be negative")
    hra, da = 0.20 * basic, 0.10 * basic
    overtime = overtime_hours * 250
    gross = basic + hra + da + overtime
    pf = 0.12 * basic
    tax = income_tax(gross)
    net = gross - pf - tax
    return {"Employee": name, "Gross": gross, "PF": pf, "Tax": tax, "Net": net}

name = input("Employee name: ")
basic = float(input("Basic salary: "))
hours = float(input("Overtime hours: "))
for key, value in payroll(name, basic, hours).items():
    print(f'{key:10}: {value if key == "Employee" else f"Rs. {value:.2f}"}')
```

Student Tasks and Test Evidence

- Run the program for employees falling in each tax slab.
- Modify one allowance rate and verify the change.
- Explain why payroll() returns a dictionary.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Conclusion

Viva Questions

- Differentiate gross salary and net salary.
- What is a return value?
- Why are functions useful in large programs?
- How does elif differ from a separate if?
- When is ValueError raised?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Correct payroll calculations and input validation (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Calculation trail, code and salary report (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 03

Student Grade Analyzer

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO1, CO2, CO4		

Aim

Analyze marks of multiple students and produce total, percentage, grade and class-performance statistics.

Learning Objectives

- Use lists, dictionaries, loops and functions.
- Calculate individual and group-level measures.
- Present performance results in a readable form.

Software / Resources

Python 3.x, Google Colab

Concept Brief

A grade analyzer stores structured student records, applies a consistent grading rule, and summarizes performance. Mean and topper information help interpret overall class performance beyond individual grades.

Algorithm / Procedure

1. Accept or define student names and subject marks.
2. Validate each mark in the range 0 to 100.
3. Calculate total, percentage and grade for each student.
4. Identify the topper and class average.
5. Display the individual and summary reports.

Reference Implementation

```
def grade(percent):
    if percent >= 85: return "A"
    if percent >= 70: return "B"
    if percent >= 55: return "C"
    if percent >= 40: return "D"
    return "F"

students = [
    {"name": "Aarav", "marks": [78, 84, 91]},
    {"name": "Diya", "marks": [88, 93, 86]},
    {"name": "Kabir", "marks": [62, 70, 68]}
]
results = []
for student in students:
    if any(mark < 0 or mark > 100 for mark in student["marks"]):
        raise ValueError("Marks must be between 0 and 100")
    total = sum(student["marks"])
    percent = total / len(student["marks"])
    results.append({"name": student["name"], "total": total,
                  "percentage": percent, "grade": grade(percent)})

for row in results:
    print(row)
print("Topper:", max(results, key=lambda x: x["percentage"])["name"])
print("Class average:", sum(r["percentage"] for r in results) / len(results))
```

Student Tasks and Test Evidence

- Add two student records and rerun the analysis.
- Display the number of students in each grade.
- Test the mark-validation rule.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Conclusion

Viva Questions

- Why is a list of dictionaries suitable here?
- What does any() return?
- How does max() use a key function?
- What is the difference between total and percentage?
- Which data are mutable in this program?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Accurate grade analysis and summary statistics (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Data structure, test cases and performance report (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 04

Inventory Tracker

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO1, CO2		

Aim

Build a menu-driven inventory system to add, update, delete, search and display stock items.

Learning Objectives

- Apply dictionaries for key-value storage.
- Implement CRUD-style operations with functions.
- Handle missing items and invalid quantities.

Software / Resources

Python 3.x, VS Code

Concept Brief

An inventory record commonly associates a unique item code with fields such as name, quantity and price. Dictionary lookup provides efficient access, while modular functions keep operations clear and reusable.

Algorithm / Procedure

1. Create an empty inventory dictionary.
2. Display a menu repeatedly until Exit is selected.
3. Perform add, update, delete, search or display operations.
4. Validate item code and quantity for each operation.
5. Show the updated inventory after a successful transaction.

Reference Implementation

```
inventory = {}

def add_item(code, name, qty, price):
    if code in inventory: raise ValueError("Item code already exists")
    inventory[code] = {"name": name, "quantity": qty, "price": price}

def update_stock(code, change):
    if code not in inventory: raise KeyError("Item not found")
    new_qty = inventory[code]["quantity"] + change
    if new_qty < 0: raise ValueError("Insufficient stock")
    inventory[code]["quantity"] = new_qty

def show_items():
    for code, item in inventory.items():
        value = item["quantity"] * item["price"]
        print(code, item["name"], item["quantity"], f"Rs. {value:.2f}")

add_item("I101", "Keyboard", 10, 750)
add_item("I102", "Mouse", 15, 400)
update_stock("I101", -2)
show_items()
# Extend with input menu for delete, search and repeated operations.
```

Student Tasks and Test Evidence

- Complete the menu for all five operations.
- Calculate total inventory value.
- Display low-stock items having quantity below five.

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Inventory operations and validation (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Menu logic, code organization and output (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 05

Log File Analyzer

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO2, CO4		

Aim

Generate application logs, analyze log levels, identify errors and export a summary report to CSV.

Learning Objectives

- Create structured log records.
- Read and parse a text file safely.
- Aggregate log levels and export CSV results.

Software / Resources

Python logging module, CSV, VS Code

Concept Brief

Logging records events with timestamps and severity levels such as INFO, WARNING and ERROR. A log analyzer transforms raw operational records into counts and focused error reports that support troubleshooting.

Algorithm / Procedure

1. Configure the logging module to write structured entries.
2. Generate representative INFO, WARNING and ERROR events.
3. Read the log file line by line.
4. Count occurrences of each level and collect error messages.
5. Write summary counts to a CSV file and display detected errors.

Reference Implementation

```
import logging, csv
from collections import Counter

logging.basicConfig(filename="app.log", level=logging.INFO,
                    format="%(asctime)s|%(levelname)s|%(message)s",
                    filemode="w")
logging.info("Application started")
logging.warning("Memory usage above 75 percent")
logging.error("Database connection failed")
logging.info("Application stopped")

counts = Counter()
errors = []
with open("app.log", encoding="utf-8") as file:
    for line in file:
        parts = line.strip().split("|", 2)
        if len(parts) == 3:
            level, message = parts[1], parts[2]
            counts[level] += 1
            if level == "ERROR": errors.append(message)

with open("log_summary.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["Log Level", "Count"])
    writer.writerows(counts.items())
print(dict(counts)); print("Errors:", errors)
```

Student Tasks and Test Evidence

- Add DEBUG and CRITICAL events and update the analyzer.
- Use tell() to record the file pointer before and after one read.

- Test the analyzer with one malformed log line.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

[illegible]

Conclusion

Viva Questions

- What is the purpose of the logging module?
- Name common logging levels.
- Why is `newline=""` used for CSV writing?
- What do `tell()` and `seek()` do?
- Why is malformed-line handling needed?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Log parsing, aggregation and CSV generation (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
File evidence, exception handling and summary report (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 06

Banking Account Management System

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO2, CO5		

Aim

Implement an object-oriented banking application with deposit, withdrawal, balance inquiry and transaction history.

Learning Objectives

- Create classes, objects, constructors and methods.
- Apply encapsulation and validation.
- Maintain a transaction history for each account.

Software / Resources

Python OOP, VS Code

Concept Brief

Object-oriented programming combines data and behavior in a class. Encapsulation protects the balance from uncontrolled modification, while methods enforce consistent rules for every transaction.

Algorithm / Procedure

1. Define a BankAccount class and initialize account data.
2. Provide methods for deposit, withdrawal and balance inquiry.
3. Reject non-positive amounts and withdrawals beyond balance.
4. Append each successful transaction to history.
5. Create objects and test every method.

Reference Implementation

```
class BankAccount:
    def __init__(self, account_no, holder, opening_balance=0):
        self.account_no = account_no
        self.holder = holder
        self.__balance = opening_balance
        self.transactions = [("OPEN", opening_balance)]

    def deposit(self, amount):
        if amount <= 0: raise ValueError("Amount must be positive")
        self.__balance += amount
        self.transactions.append(("DEPOSIT", amount))

    def withdraw(self, amount):
        if amount <= 0: raise ValueError("Amount must be positive")
        if amount > self.__balance: raise ValueError("Insufficient balance")
        self.__balance -= amount
        self.transactions.append(("WITHDRAW", amount))

    def balance(self):
        return self.__balance

account = BankAccount("AC101", "Riya", 5000)
account.deposit(1500)
account.withdraw(700)
print("Balance:", account.balance())
print("History:", account.transactions)
```

Student Tasks and Test Evidence

- Create two account objects and perform transactions.
- Add a transfer method between two accounts.

- Create a SavingsAccount subclass with an interest method.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

[illegible]

Conclusion

Viva Questions

- What is a class?
- What is the role of `__init__()`?
- Why is `balance` written with two leading underscores?
- Differentiate instance variable and class variable.
- How can inheritance extend this program?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
OOP design and banking operations (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Class diagram/logic, code and transaction tests (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 07

Customer Record Management App

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO3, CO5	/10	

Aim

Develop a database-integrated application that performs create, read, update and delete operations on customer records.

Learning Objectives

- Connect Python with a relational database.
- Execute parameterized CRUD queries.
- Apply transaction handling and input validation.

Software / Resources

Python, SQLite, VS Code

Concept Brief

SQLite is a file-based relational database included with Python. Parameterized queries keep data separate from SQL instructions and reduce injection risks. Transactions ensure that changes are committed only when operations succeed.

Algorithm / Procedure

1. Connect to the database and create the customer table if absent.
2. Insert a customer using a parameterized query.
3. Read and display customer records.
4. Update or delete a selected record.
5. Commit successful changes and close the connection.

Reference Implementation

```
import sqlite3

with sqlite3.connect("customers.db") as con:
    cur = con.cursor()
    cur.execute("""CREATE TABLE IF NOT EXISTS customer(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL, email TEXT UNIQUE, city TEXT)""")
    cur.execute("INSERT OR IGNORE INTO customer(name,email,city) VALUES(?,?,?)",
        ("Meera", "meera@example.com", "Rajkot"))
    cur.execute("UPDATE customer SET city=? WHERE email=?",
        ("Ahmedabad", "meera@example.com"))
    for row in cur.execute("SELECT id,name,email,city FROM customer"):
        print(row)
    # Example delete:
    # cur.execute("DELETE FROM customer WHERE id=?", (customer_id,))
    con.commit()
```

Student Tasks and Test Evidence

- Build a menu for insert, search, update and delete.
- Test the UNIQUE constraint with a duplicate email.
- Display customers filtered by city.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

[illegible]

Conclusion

Viva Questions

- What is CRUD?
- Why are parameterized queries recommended?
- What is a primary key?
- What is the purpose of commit()?
- How does a context manager help database handling?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Database connection and correct CRUD operations (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Schema, queries, code and database output (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 08

Weather API Dashboard

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO3, CO4		

Aim

Fetch live weather data from a public REST API, parse the JSON response and display a simple dashboard.

Learning Objectives

- Send an HTTP GET request with parameters.
- Parse nested JSON safely.
- Display meaningful weather information and handle network errors.

Software / Resources

Python requests, Tkinter, public weather API

Concept Brief

A REST API exposes resources through HTTP. A GET request retrieves data, commonly encoded as JSON. Robust clients check response status, set a timeout and handle missing fields or connectivity failures.

Algorithm / Procedure

1. Accept latitude and longitude from the user.
2. Send a GET request to the weather endpoint with a timeout.
3. Check the response status and parse JSON.
4. Extract current temperature and wind speed.
5. Display the report or a meaningful error message.

Reference Implementation

```
import requests

def current_weather(latitude, longitude):
    url = "https://api.open-meteo.com/v1/forecast"
    params = {"latitude": latitude, "longitude": longitude,
              "current": "temperature_2m,wind_speed_10m"}
    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    current = response.json()["current"]
    return current["temperature_2m"], current["wind_speed_10m"]

try:
    temperature, wind = current_weather(22.3039, 70.8022)
    print(f"Temperature: {temperature} degree C")
    print(f"Wind speed: {wind} km/h")
except (requests.RequestException, KeyError) as error:
    print("Weather data unavailable:", error)
# Extend by displaying the values in a Tkinter dashboard.
```

Student Tasks and Test Evidence

- Test the request for a second location.
- Display the last update time from the response.
- Create a Tkinter interface for latitude, longitude and results.

Test case / input	Expected result	Actual result	Status
1.			
2.			
3.			

Observation

[illegible]

Conclusion

Viva Questions

- What is a REST API?
- What is JSON?
- Why is a request timeout important?
- What does `raise_for_status()` do?
- Differentiate GET and POST.

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
API request, JSON parsing and dashboard output (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Request parameters, error handling and result evidence (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 09

Email Automation Tool

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO3, CO5	/10	

Aim

Build a configurable email automation tool for scheduled reminders while protecting login credentials and recipient privacy.

Learning Objectives

- Construct an email message using Python.
- Use environment variables for credentials.
- Schedule a reusable sending function and apply safe-testing practices.

Software / Resources

Python smtplib, email.message, schedule

Concept Brief

SMTP is used to transfer email. Automation must protect passwords, verify recipients and avoid unintended bulk delivery. Environment variables prevent credentials from being stored directly in source code. A dry-run mode should be tested before live sending.

Algorithm / Procedure

1. Read sender address and application password from environment variables.
2. Build the message with subject, sender, recipient and body.
3. When dry-run is enabled, print the message instead of sending it.
4. Otherwise establish a secure SMTP connection and send.
5. Register the function with a schedule and run pending tasks.

Reference Implementation

```
import os, smtplib
from email.message import EmailMessage

def send_reminder(recipient, dry_run=True):
    message = EmailMessage()
    message["Subject"] = "Python Lab Reminder"
    message["From"] = os.getenv("LAB_EMAIL", "teacher@example.com")
    message["To"] = recipient
    message.set_content("Please complete the scheduled Python lab task.")
    if dry_run:
        print(message)
        return
    password = os.environ["LAB_EMAIL_APP_PASSWORD"]
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
        server.login(message["From"], password)
        server.send_message(message)

send_reminder("student@example.com", dry_run=True)
# After faculty approval, schedule safely:
# import schedule, time
# schedule.every().day.at("16:00").do(send_reminder, recipient="...", dry_run=False)
```

Student Tasks and Test Evidence

- Run only in dry-run mode and verify all message fields.
- Add recipient validation and an attachment-size check.
- Explain the safeguards required before enabling bulk delivery.

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Message construction, secure configuration and scheduling logic (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Dry-run evidence, safeguards and code explanation (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 10

Sales Analytics Dashboard

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO4, CO5	/10	

Aim

Analyze a sales dataset and create charts and summary statistics that communicate product and monthly trends.

Learning Objectives

- Import and clean tabular data.
- Group and aggregate sales measures.
- Create correctly labelled charts and interpret results.

Software / Resources

Python, Pandas, NumPy, Matplotlib

Concept Brief

Pandas DataFrames organize tabular records and support filtering, missing-value handling and grouping. Matplotlib converts aggregate values into charts. A useful dashboard combines accurate measures, readable labels and a short interpretation.

Algorithm / Procedure

1. Load or create a sales dataset.
2. Inspect data types and missing values.
3. Clean records and calculate revenue where required.
4. Group revenue by product and month.
5. Plot charts, save the figure and state key findings.

Reference Implementation

```
import pandas as pd
import matplotlib.pyplot as plt

data = {"Month": ["Jan", "Jan", "Feb", "Feb", "Mar", "Mar"],
        "Product": ["A", "B", "A", "B", "A", "B"],
        "Units": [20, 15, 25, 18, 30, 22],
        "Price": [500, 750, 500, 750, 500, 750]}
df = pd.DataFrame(data)
df["Revenue"] = df["Units"] * df["Price"]
monthly = df.groupby("Month", sort=False)["Revenue"].sum()
product = df.groupby("Product")["Revenue"].sum()
print(df); print("Total revenue:", df["Revenue"].sum())

fig, axes = plt.subplots(1, 2, figsize=(9, 4))
monthly.plot(kind="line", marker="o", ax=axes[0], title="Monthly Revenue")
product.plot(kind="bar", ax=axes[1], title="Revenue by Product")
for ax in axes: ax.set_ylabel("Revenue (Rs.)"); ax.grid(axis="y", alpha=.3)
plt.tight_layout(); plt.savefig("sales_dashboard.png", dpi=200); plt.show()
```

Student Tasks and Test Evidence

- Import equivalent data from a CSV file.
- Introduce one missing value and apply a justified cleaning method.
- Write two evidence-based observations from the charts.

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
Data cleaning, analysis and correct visualization (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Dataset evidence, labelled charts and interpretation (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 11

Attendance Management System

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO3, CO5	/10	

Aim

Develop an attendance system with a GUI, database storage, attendance marking and report generation.

Learning Objectives

- Integrate Tkinter with SQLite.
- Enforce one attendance record per student per date.
- Generate an attendance report.

Software / Resources

Python, SQLite, Tkinter

Concept Brief

A database-backed GUI separates interface events from persistent records. A UNIQUE constraint on student and date prevents duplicate attendance. Parameterized queries and validation protect data integrity.

Algorithm / Procedure

1. Create student and attendance tables.
2. Build fields for student ID, date and status.
3. Validate values and insert attendance using a parameterized query.
4. Prevent duplicate entries using a database constraint.
5. Query and display or export the attendance report.

Reference Implementation

```
import sqlite3, tkinter as tk
from tkinter import messagebox

con = sqlite3.connect("attendance.db")
con.execute("""CREATE TABLE IF NOT EXISTS attendance(
    student_id TEXT, attend_date TEXT, status TEXT,
    UNIQUE(student_id, attend_date)""")

def mark_attendance():
    try:
        con.execute("INSERT INTO attendance VALUES(?,?,?)",
            (student_id.get().strip(), attend_date.get().strip(), status.get()))
        con.commit(); messagebox.showinfo("Saved", "Attendance recorded")
    except sqlite3.IntegrityError:
        messagebox.showerror("Duplicate", "Attendance already marked")

root = tk.Tk(); root.title("Attendance Management")
student_id, attend_date, status = tk.StringVar(), tk.StringVar(), tk.StringVar(value="Present")
for row, (label, variable) in enumerate(((("Student ID", student_id), ("Date YYYY-MM-DD", attend_date))):
    tk.Label(root, text=label).grid(row=row, column=0, padx=8, pady=6)
    tk.Entry(root, textvariable=variable).grid(row=row, column=1)
tk.OptionMenu(root, status, "Present", "Absent").grid(row=2, column=1)
tk.Button(root, text="Save", command=mark_attendance).grid(row=3, column=0, columnspan=2)
root.mainloop(); con.close()
```

Student Tasks and Test Evidence

- Add a student master table and validate student IDs.
- Create a date-wise attendance report.
- Calculate attendance percentage for a selected student.

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
GUI-database integration and report accuracy (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Schema, validation, screenshots and result (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

EXPERIMENT 12

Helpdesk Ticketing Tool (Capstone)

Date performed	Date submitted	Mapped CO(s)	Marks	Faculty signature
		CO3, CO5		

Aim

Create a web-based helpdesk system to log, assign, update and resolve support tickets.

Learning Objectives

- Build Flask routes for ticket workflow operations.
- Store tickets in MySQL using parameterized queries.
- Apply status rules, validation and collaborative development practices.

Software / Resources

Python, Flask, MySQL

Concept Brief

A helpdesk ticket moves through defined states such as Open, Assigned, In Progress and Resolved. Flask connects HTTP routes to Python functions; MySQL provides persistent multi-user storage. Clear workflow rules and audit fields support accountability.

Algorithm / Procedure

1. Create the ticket database and table.
2. Configure a Flask application and database connection.
3. Implement routes to create, list, assign and update tickets.
4. Validate ticket fields and use parameterized SQL.
5. Test the full workflow and document team responsibilities.

Reference Implementation

```

from flask import Flask, request, jsonify
import mysql.connector

app = Flask(__name__)

def db():
    return mysql.connector.connect(host="localhost", user="root",
        password="YOUR_LOCAL_PASSWORD", database="helpdesk")

@app.post("/tickets")
def create_ticket():
    data = request.get_json()
    if not data or not data.get("title"):
        return {"error": "title is required"}, 400
    con = db(); cur = con.cursor()
    cur.execute("INSERT INTO ticket(title,description,status) VALUES(%s,%s,%s)",
        (data["title"], data.get("description", ""), "Open"))
    con.commit(); ticket_id = cur.lastrowid; cur.close(); con.close()
    return {"id": ticket_id, "status": "Open"}, 201

@app.get("/tickets")
def list_tickets():
    con = db(); cur = con.cursor(dictionary=True)
    cur.execute("SELECT * FROM ticket ORDER BY id DESC")
    rows = cur.fetchall(); cur.close(); con.close()
    return jsonify(rows)

if __name__ == "__main__":
    app.run(debug=True)
# Add PUT routes for assignment/status updates and test them in Postman.

```

Student Tasks and Test Evidence

- Create the MySQL ticket table with timestamps and an assignee field.
- Add parameterized routes for assignment and status update.
- Test valid and invalid workflow cases in Postman and prepare a team contribution record.

Test case / input	Expected result	Actual result	Status
1.			Pass / Fail
2.			Pass / Fail
3.			Pass / Fail

Observation

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Conclusion

Viva Questions

- What is a Flask route?
- Why are HTTP status codes important?
- What is SQL injection?
- Why should debug mode be disabled in production?
- How does the ticket workflow support teamwork?

Experiment-Specific Rubric (10 Marks)

Criterion	Full performance	Developing performance	Beginning / absent
On-time completion and submission (2)	Completed and submitted during/on the scheduled lab date.	Completed with minor delay or one reminder.	Submitted late/incomplete; 0 if not submitted.
End-to-end ticket workflow and functional integration (3)	Complete, correct and independently executable solution; all required cases work.	Mostly correct; minor defects or limited testing.	Partly working solution; 0 if no executable work.
Design, API tests, documentation and contribution evidence (2)	Clear algorithm, readable code, meaningful output and valid result statement.	Record is present but has minor omissions or weak explanation.	Major omissions/poor documentation; 0 if absent.
Viva voce (3)	Explains concepts, code and output accurately and confidently.	Answers basic questions but needs prompts.	Limited understanding; 0 for no meaningful response.

Faculty feedback: _____

Appendix A. Suggested Software Setup

Purpose	Package / tool	Typical installation
Core programming	Python 3.x, VS Code or Jupyter	Install Python from python.org; verify with <code>python --version</code>
HTTP requests	requests	<code>pip install requests</code>
Scheduling	schedule	<code>pip install schedule</code>
Analytics	pandas, numpy, matplotlib	<code>pip install pandas numpy matplotlib</code>
Web application	Flask	<code>pip install flask</code>
MySQL connection	mysql-connector-python	<code>pip install mysql-connector-python</code>

Appendix B. Submission Checklist

- Aim and mapped CO(s) are recorded.
- Algorithm matches the implemented program.
- Program executes for normal, boundary and invalid inputs.
- Source code is readable and credentials are excluded.
- Screenshots/output evidence is attached or pasted.
- Result statement is written in the student's own words.
- External datasets, APIs and AI-assisted contributions are acknowledged.
- Faculty marks, feedback and signature are completed.

Appendix C. Source Basis

Course title, code, credits, outcomes, experiment list and lab assessment structure are based on the approved Programming with Python (01AP0303) syllabus and the supplied comprehensive student guideline. University and programme statements are based on Marwadi University's official pages. Programme Outcomes follow NBA GAPC V4.0. Sources verified on 18 September 2026.

- <https://www.marwadiuniversity.ac.in/about-us/>
- <https://www.marwadiuniversity.ac.in/btech-in-computer-science-engineering/>
- https://www.nbaind.org/files/2-SAR-UG-EG-T2-14-1-2025_Format_20250115122255.pdf